Assignment #4 — Facade Pattern

Online order processing

Course: ShP-2216 — Software Design Patterns (OP 6B06102)
Institution: Astana IT University — School of Computer Engineering
Student: Adilzhan Kuandykov
Group: SE-2526
Language: Java (JDK 17)
Repository: https://github.com/adilzhandev/assignment-4-facade-pattern

---

1. Introduction

This project is a small online shop. It implements Option B of the assignment in Java 17. An order needs four services: inventory, payment, shipping, and notification. OrderFacade connects these services through two methods: placeOrder(cart, customer) and cancelOrder(orderId).

I use the Facade pattern for three reasons.

- The steps have to run in a particular order. The shop checks and reserves the stock before taking payment. It registers a shipment only after payment succeeds.
- A failed step must not leave the previous steps unfinished. If payment is declined, the reserved stock must be released. If shipping fails, the payment must also be refunded.
- The client should not repeat these rules. Main sends one request to the facade, and the facade makes the service calls and handles cleanup.

During placement, the inventory service validates the whole cart and calculates the total. The facade then charges the customer, registers the shipment, and records a notification. Only after these steps succeed does it store an Order with status PLACED. Cancellation restores the stock and balance and changes the status to CANCELLED. Repeating the cancellation does not restore the same resources twice.

The implementation uses maps and lists in memory. Prices and balances are stored in whole cents. No real payment, delivery, or email is sent, and the data is reset when the program stops. This keeps the example focused on the pattern and makes it easy to demonstrate.

2. UML class diagram

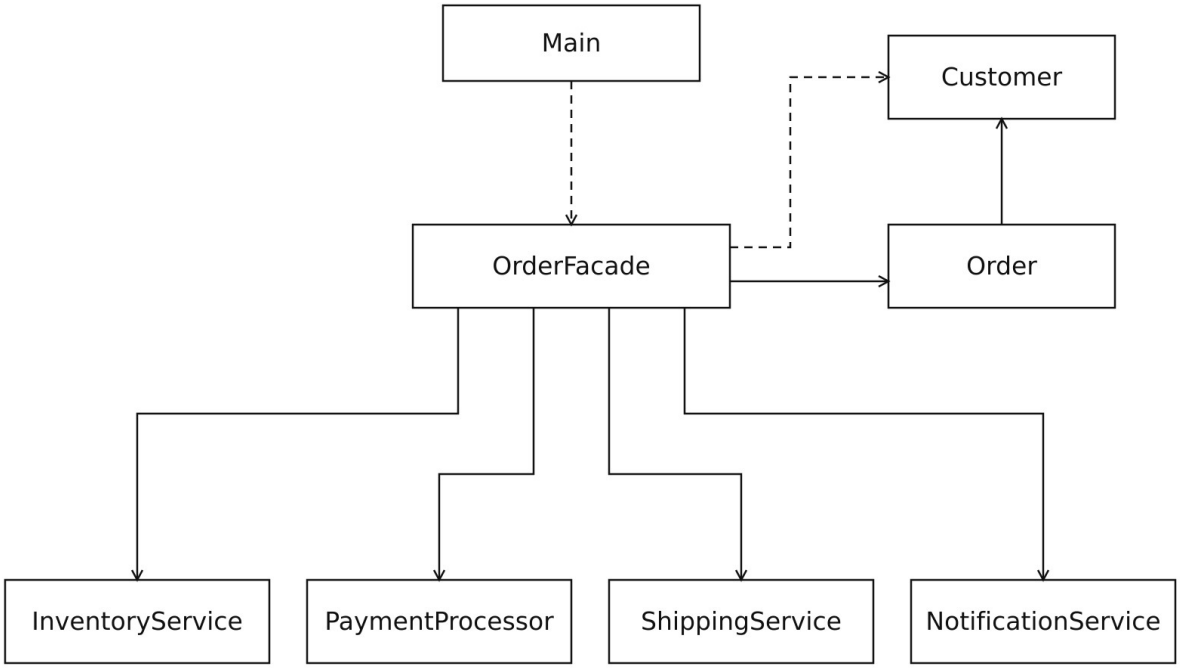


Figure 1 — Class names and relationships in the Facade implementation

The table below shows which class has which role in the pattern.

Role in the pattern	Class in this project
Client	Main
Facade	OrderFacade
Inventory subsystem	InventoryService
Payment subsystem	PaymentProcessor
Shipping subsystem	ShippingService
Notification subsystem	NotificationService
Data records	Customer, Order

Main calls OrderFacade and uses Customer and Order as data. The facade coordinates the four services and stores successful orders. Solid arrows show stored references; dashed arrows show dependencies. The diagram shows the main relationships and leaves out attributes, methods, and the status enumeration.

None of the services has a reference to OrderFacade. An advanced client can create the services directly and coordinate them itself. The facade provides a simpler way to use them without making them inaccessible.

3. Clean Code

I use six principles in this implementation. Each example comes from the current source code. The explanation below the excerpt describes the reason for the design.

3.1 Meaningful names

The names describe what the client wants to do. There is no need to know the individual services to understand the request.

Code — OrderFacade.java

```
public Order placeOrder(Map<String, Integer> cart, Customer customer) {
```

placeOrder describes the action, while cart and customer describe its inputs. cancelOrder(orderId) is the matching operation. The name totalCents also states the unit of the amount, so a reader does not have to guess whether it is a decimal price.

3.2 The facade has one responsibility

OrderFacade coordinates the workflow. Each service performs its own operation.

Code — OrderFacade.java

```
payments.charge(orderId, customer.email(), totalCents);
shipping.ship(orderId, customer.address());
notifications.send(customer.email(), "Order " + orderId + " placed");
```

The facade decides when these calls happen and passes the same order ID to each service. It does not calculate stock quantities, change payment balances directly, or store shipment addresses itself. Those details stay inside the service that owns them.

3.3 Subsystem details stay out of the public API

Code — OrderFacade.java

```
public OrderFacade(Map<String, Long> prices, Map<String, Integer> stock,
                   Map<String, Long> customerBalances) {
    this(new InventoryService(prices, stock), new PaymentProcessor(customerBalances),
         new ShippingService(), new NotificationService());
}
```

The public constructor accepts prices, stock, and customer balances as ordinary data. It creates the services internally. The business methods return an Order, and there are no service getters. A separate package-private constructor accepts services for testing, but the normal client does not need it.

3.4 Small and focused subsystem classes

Each service owns one part of the order process. For example, ShippingService stores delivery addresses and provides simple operations for shipment records.

Code — service/ShippingService.java

```
public void cancel(String orderId) {
    shipments.remove(orderId);
}

public boolean hasShipment(String orderId) {
    return shipments.containsKey(orderId);
}
```

This class has no inventory or payment logic. InventoryService manages stock and reservations, PaymentProcessor manages balances and recorded payments, and NotificationService validates and stores messages. A change to one of these rules has a clear place in the project.

3.5 Proper error handling

The facade undoes completed work when a placement step fails. The cleanup order is the reverse of the processing order.

Code — OrderFacade.java

```
} catch (RuntimeException failure) {
    shipping.cancel(orderId);
    payments.refund(orderId);
    inventory.release(orderId);
```

A missing shipment or payment record makes its cleanup call do nothing. Therefore, a payment rejection still reaches inventory.release and returns the stock. The next line throws an IllegalStateException with the original failure as its cause. InventoryService checks all cart items before changing stock, so a failed validation cannot leave only part of a cart reserved.

3.6 Defensive copying

Code — model/Order.java

```
public record Order(String id, Customer customer, Map<String, Integer> items,
                    long totalCents, Status status) {
    public Order {
        items = Map.copyOf(items);
    }
```

Order is a record, and Map.copyOf protects its item map. Changing the original cart does not change the stored order, and order.items() cannot be edited. This is important when cancelling an order: the quantities restored must be the same quantities that were reserved. The facade and inventory service also copy their input maps.

4. Conclusion

The facade hides service creation, the order of calls, order ID creation and propagation, and the cleanup rules for failed placement. Main can place or cancel an order through one object. The service classes remain public and can still be used directly.

The main benefits of this design are:

- The client describes a business operation instead of repeating four service calls and the error handling around them.
- Each service keeps its own state and rules. The facade only coordinates how the services work together.
- The test suite can inspect each service and simulate a failure at a particular step. This makes it possible to check that stock, payments, and shipments are restored.

There are also limits. The application is designed for one thread and does not persist its data. Cleanup assumes that the in-memory cancel, refund, and release methods complete normally. Cancellation stores the CANCELLED state before recording its notification; a failure in a replacement notification service would not undo that completed cancellation.

5. Repository

Source code: https://github.com/adilzhandev/assignment-4-facade-pattern

The repository contains the Java source in src/main/java/shop, tests in src/test/java/shop, and the report and UML files in docs. The README explains how to run the project with JDK 17. From the project root, use:

```
make run
make test
```

The console demonstration buys the only keyboard, cancels the order, rejects Bob's payment, and then buys the keyboard again for Alice. The last purchase succeeds only when Alice has been refunded and Bob's failed payment has released the stock.

All 12 tests pass with Java 17, all lint checks enabled, and warnings treated as errors. They cover successful orders, payment, shipping and notification failures, atomic reservation, cancellation, invalid inputs, immutable carts, independent services, and the public facade constructor. The test runner finishes with:

```
Passed 12 tests.
```

The code excerpts in this report refer to implementation commit e55dd96.